

PLANO DE P&D CONTÍNUO E ARQUITETURA EVOLUTIVA

Este documento consolida (1) um **plano de Pesquisa & Desenvolvimento contínuo** e (2) um **diagrama arquitetural evolutivo** para crescimento sustentado de um **agente autônomo local**, com foco em robustez, aprendizado contínuo, baixa interferência operacional e governança forte.

PARTE I — PLANO DE P&D CONTÍNUO (12–18 MESES)

Fase A — Consolidação Cognitiva (0–2 meses)

Objetivo: estabilizar o agente como sistema previsível.

- Congelar arquitetura agent-centric (LangGraph)
- Separar definitivamente:
 - Planejamento (LLM)
 - Execução (runtime tipado)
 - Memória (RAG + SQL)
- Introduzir **schemas formais** para todas as ações do agente
- Métricas iniciais:
 - taxa de falha por tool
 - loops cognitivos
 - latência por estado

Resultado esperado: - Agente confiável - Zero execução arbitrária

Fase B — Memória Avançada e Reflexão (2–4 meses)

Objetivo: transformar histórico em aprendizado real.

- Implementar **memória hierárquica**:
 - curta (contextual)
 - média (tarefas)
 - longa (experiência)
- Criar módulo de **reflexão pós-execução**:
 - o que funcionou
 - o que falhou
 - quais ferramentas foram úteis
- Indexar reflexões em Qdrant

Resultado esperado: - Redução de erros repetidos - Planejamento mais eficiente

Fase C — Avaliação Interna e Auto-Crítica (4–6 meses)

Objetivo: reduzir dependência de validação externa.

- Criar **avaliador interno** (LLM menor ou regras simbólicas)
- Pontuar:
 - respostas
 - uso de ferramentas
 - decisões de ingestão
- Integrar avaliação como gate obrigatório

Resultado esperado: - Aumento de precisão - Rejeição automática de respostas fracas

Fase D — Aprendizado Contínuo Controlado (6–9 meses)

Objetivo: permitir evolução sem contaminação.

- Pipeline ALAS-style:
 - detectar gaps
 - buscar dados
 - validar qualidade
 - atualizar base
- Nunca ajustar pesos do LLM principal
- Aprendizado via:
 - memória
 - ferramentas
 - heurísticas

Resultado esperado: - Conhecimento atualizado - Persona estável

Fase E — Meta-Aprendizado (9–12 meses)

Objetivo: o agente aprende COMO aprender.

- Registrar estratégias de sucesso
- Criar ranking de abordagens
- Ajustar políticas de busca, RAG e tools

Resultado esperado: - Menos tentativas - Mais eficiência cognitiva

Fase F — Multi-Agentes Especializados (12–18 meses)

Objetivo: escalar capacidade sem escalar risco.

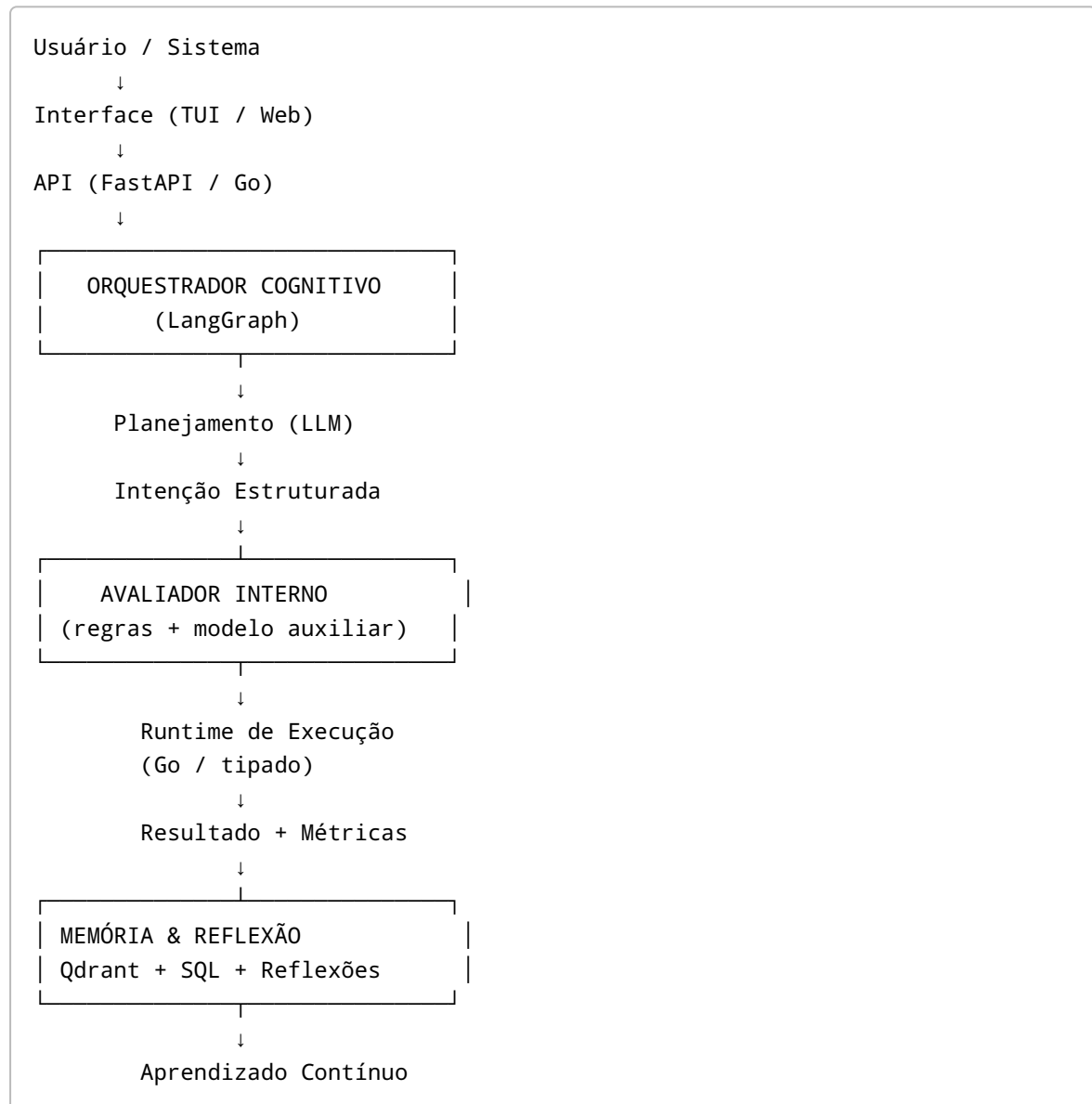
- Sub-agentes especializados:
 - pesquisa
 - validação
 - execução

- Orquestrador central
- Consenso mínimo para ações críticas

Resultado esperado: - Robustez - Paralelismo seguro

PARTE II — DIAGRAMA ARQUITETURAL EVOLUTIVO

Visão Geral (Estado Maduro)



Separação Estrutural Crítica

- LLM nunca executa código
- Runtime nunca planeja
- Memória nunca decide
- Interface nunca controla diretamente

PARTE III — PRINCÍPIOS DE DESIGN PARA EVOLUÇÃO SEGURA

1. Limitação consciente > poder irrestrito
 2. Tipos e contratos antes de inteligência
 3. Observabilidade antes de autonomia
 4. Aprender por experiência, não por peso
 5. Evoluir ferramentas, não persona
-

PARTE IV — RESULTADO ESPERADO

Um agente que: - Aprende continuamente sem se corromper - Evolui comportamento sem alterar identidade - Escala capacidade sem escalar risco - Opera como sistema técnico previsível

Este plano permite crescimento por **anos**, não apenas por versões.

CHECKLIST TÉCNICO EXECUTÁVEL — IMPLEMENTAÇÃO DO AGENTE AUTÔNOMO

FASE 0 — Preparação e Fundamentos

- [] Congelar hardware alvo (RTX 4070 12GB validada)
- [] Instalar drivers NVIDIA + CUDA compatível
- [] Definir SO base (Linux recomendado)
- [] Criar repositório monorepo
- [] Definir política de logs e auditoria

FASE 1 — Infraestrutura Base

- [] Instalar Ollama local
- [] Testar LLM (Qwen2.5 7B Q5_K_M $\leq$ 10.5GB VRAM)
- [] Subir Qdrant (Docker)
- [] Subir SearXNG (Docker, acesso restrito)
- [] Configurar Kiwix (ZIM offline)
- [] Monitorar VRAM com nvidia-smi

FASE 2 — API e Núcleo do Sistema

- [] Criar FastAPI base
- [] Endpoint /chat
- [] Endpoint /tools
- [] Endpoint /memory
- [] Endpoint /ingestion
- [] Middleware de auditoria

FASE 3 — RAG Funcional

- ☐ Pipeline de ingestão local
- ☐ Chunking (600 / overlap 100)
- ☐ Embeddings (nomic)
- ☐ Indexação Qdrant
- ☐ Retrieval Top-K
- ☐ Citação obrigatória

FASE 4 — Busca Externa Controlada

- ☐ Integração SearXNG
- ☐ Pré-filtragem de resultados
- ☐ Normalização de conteúdo
- ☐ Quarentena de documentos

FASE 5 — Score de Confiança

- ☐ Implementar critérios objetivos
- ☐ Calcular score (0-100)
- ☐ Rejeitar < 70
- ☐ Persistir justificativa

FASE 6 — Agente LangGraph

- ☐ Definir estados
- ☐ Definir transições explícitas
- ☐ Limite de ciclos
- ☐ Logs por estado

FASE 7 — Ferramentas Dinâmicas

- ☐ Contrato único de ferramentas
- ☐ Validador AST
- ☐ Allowlist de imports
- ☐ Testes unitários automáticos
- ☐ Registro versionado

FASE 8 — Runtime Seguro

- ☐ Execução isolada
- ☐ Timeout rígido
- ☐ Sem filesystem livre
- ☐ Sem rede
- ☐ Memória controlada

FASE 9 — Interface Visual

- ☐ Escolher stack (Flutter / Tauri / Web)
- ☐ Chat UI

- [] Visualizar fontes RAG
- [] Exibir score de confiança
- [] Monitorar estado do agente

FASE 10 — Hardening e Estabilidade

- [] Detecção de loops
- [] Limite de ferramentas
- [] Rollback de conhecimento
- [] Testes de stress

FASE 11 — Validação Final

- [] VRAM estável
- [] Nenhuma execução arbitrária
- [] Auditoria completa
- [] Reprodutibilidade garantida

ESPECIFICAÇÕES AVANÇADAS DO SISTEMA

A. Stack de Linguagem por Subsistema (Decisão Técnica)

A.1 Núcleo do Agente / Orquestração Cognitiva

Linguagem: Python (restrito) - Motivo: ecossistema ML maduro, LangGraph, LlamaIndex - Uso limitado a: - Planejamento - Raciocínio - Orquestração - Regras: - Sem execução dinâmica livre - Tipagem gradual (mypy) - Estrutura imutável de estados

A.2 Runtime de Ferramentas (Execução)

Linguagem: Go (recomendado) - Benefícios: - Binário único - Excelente isolamento - Sem problemas de binding como Rust - Controle de memória previsível - Função: - Executar ferramentas validadas - Sandbox lógico

A.3 Ingestão e Normalização

Linguagem: Python ou Go - Python: parsing rápido - Go: pipelines estáveis

A.4 Interface Visual

Prioridade: Flutter > Tauri > Web - Interface desacoplada - Comunicação via API

B. Política de Aprendizado Contínuo Seguro (Sem Fine-Tuning)

Princípio Central

O sistema **não aprende alterando pesos do modelo**. Aprendizado ocorre via: - Memória vetorial - Ferramentas novas - Heurísticas de score

Modos Permitidos

- Expansão de base de conhecimento
- Criação de ferramentas reutilizáveis
- Ajuste de parâmetros de busca

Modos Proibidos

- Auto fine-tuning
- Alteração de prompt system por ingestão externa
- Escrita em memória de identidade

Ciclo de Aprendizado

Observação

- Avaliação
- Score
- Persistência controlada
- Auditoria

C. Ambiente de Testes Automatizados do Agente

Tipos de Testes

C.1 Testes Cognitivos

- Perguntas ambíguas
- Detecção de loop
- Uso correto de ferramentas

C.2 Testes de Segurança

- Tentativas de execução proibida
- Prompt injection
- Conteúdo contaminado

C.3 Testes de Performance

- Latência por estado
- Consumo de VRAM
- Uso de CPU/RAM

Automação

- Execução em CI local
- Relatórios versionados
- Regressão obrigatória

D. Critérios de Maturidade do Agente

Nível 1: Agente reativo (RAG) Nível 2: Agente com ferramentas Nível 3: Agente auto-extensível Nível 4: Agente com aprendizado seguro Nível 5: Agente operacional de longo prazo

Objetivo final: **Nível 4 estável**

ESPECIFICAÇÃO DE INTERFACES — PYTHON (AGENTE) ↔ GO (RUNTIME)

1. Princípio Arquitetural

- Python **nunca executa código arbitrário**
- Go é o **único responsável por execução de ferramentas**
- Comunicação via **API local (HTTP ou Unix Socket)**
- Interface estritamente tipada (JSON Schema)

Python (LangGraph)

- Planeja
- Solicita execução
- Avalia resultado

Go Runtime

- Valida chamada
- Executa sandbox
- Retorna resultado

2. Contrato de Execução de Ferramentas

Endpoint

POST /runtime/execute

Request (Python → Go)

```
{
  "tool_name": "string",
  "tool_version": "semver",
  "input": {"key": "value"},
  "execution_id": "uuid",
  "timeout_ms": 500
}
```

Response (Go → Python)

```
{
  "execution_id": "uuid",
  "status": "success | error | timeout",
  "output": {"result": "any"},
  "logs": ["string"],
  "metrics": {
    "cpu_ms": 12,
    "memory_kb": 128
  }
}
```

3. Registro de Ferramentas

Endpoint

```
GET /runtime/tools
```

Response

```
[
  {
    "tool_name": "string",
    "version": "semver",
    "description": "string",
    "input_schema": {},
    "output_schema": {},
    "trusted": true
  }
]
```

4. Ciclo de Criação de Ferramentas

1. LLM projeta função (Python AST)
2. Validator aprova
3. Código convertido para Go (manual ou template)
4. Compilação isolada
5. Registro no runtime
6. Disponibilização para o agente

⚠ Python **não executa** a ferramenta recém-criada

5. Isolamento e Segurança no Runtime Go

- Sem acesso à rede
 - Sem filesystem arbitrário
 - Sem reflection dinâmica
 - Limite rígido de tempo
 - Limite de memória
-

6. Tipos de Erro Padronizados

Código	Descrição
RT-01	Ferramenta inexistente
RT-02	Versão inválida
RT-03	Input inválido
RT-04	Timeout
RT-05	Execução proibida

7. Benefícios Diretos desta Separação

- Elimina malformed execution em Python
- Elimina problemas de binding Rust
- Debug previsível
- Segurança verificável
- Runtime auditável